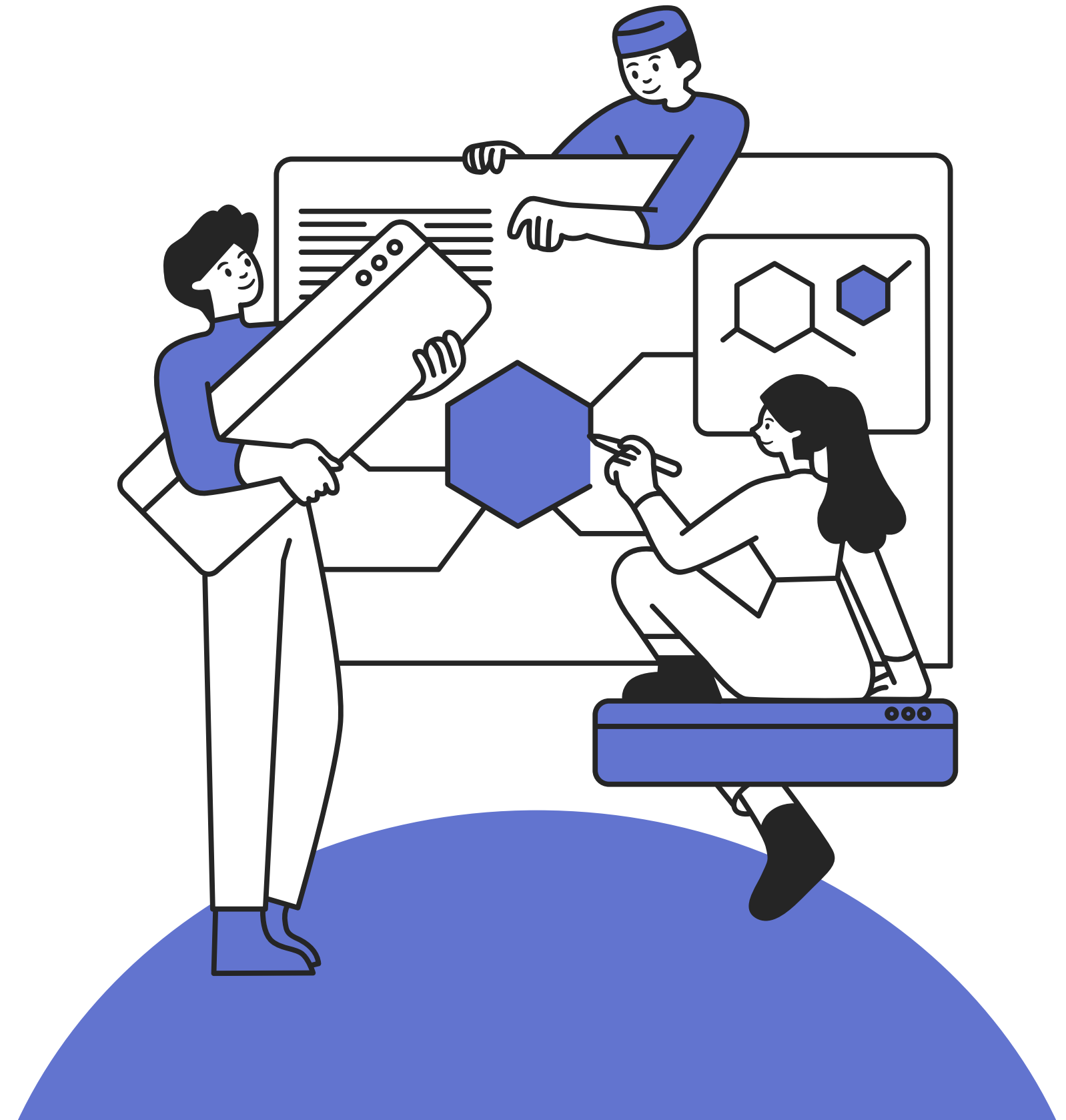


동적 계획법

이예린T



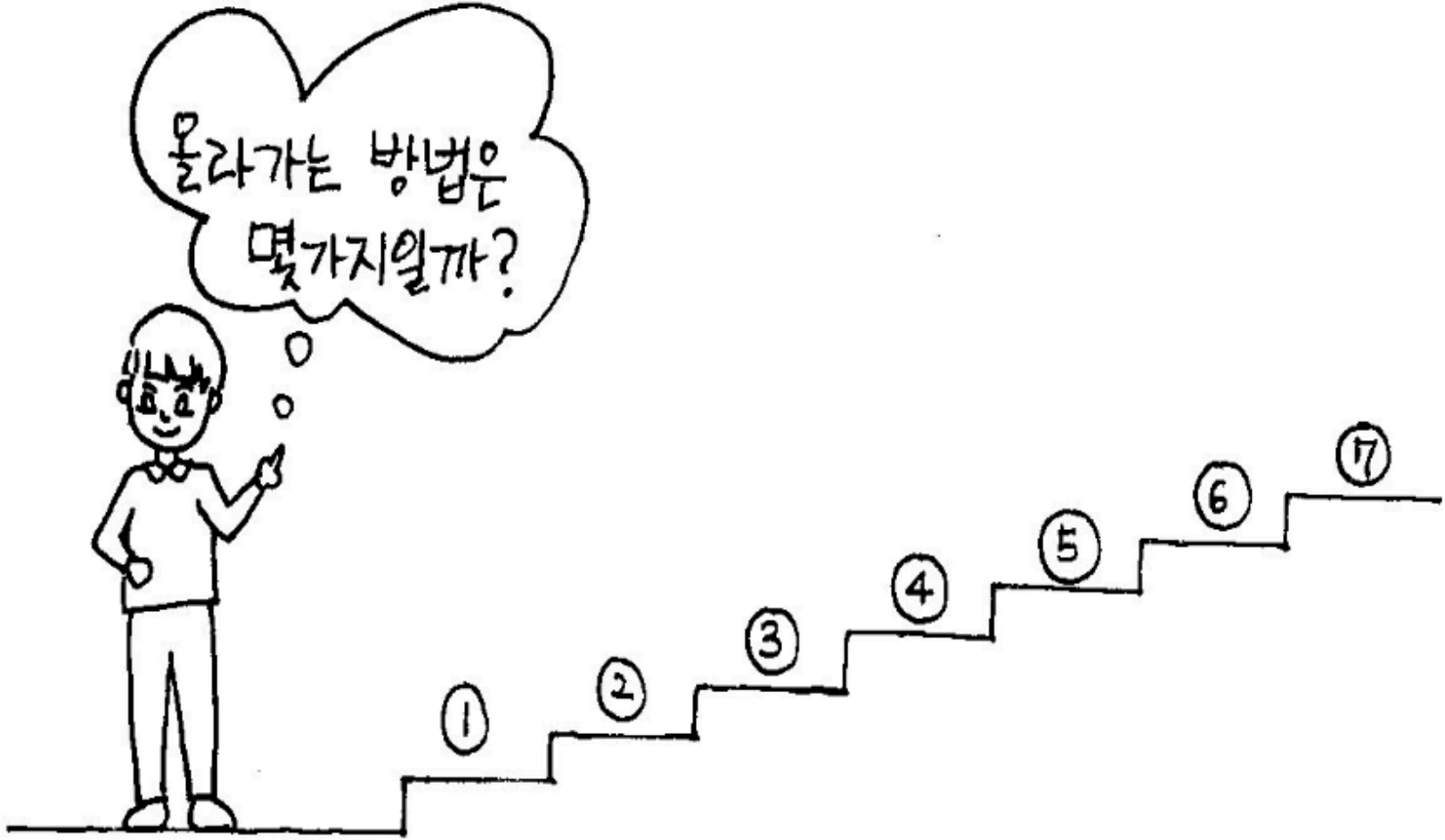
CONTENTS

- ① 동적 계획법
- ② 동적 계획법 대표 문제
- ③ 모듈별 탐구
- ④ 동적 계획법 정리

1 동적 계획법

계단오르기

철수는 계단을 오를 때 한 번에 한 계단 또는 두 계단씩 올라간다. 만약 총 4계단을 오른다면 그 방법의 수는
1+1+1+1, 1+1+2, 1+2+1, 2+1+1, 2+2 로 5가지이다.
그렇다면 총 N계단일 때 철수가 올라갈 수 있는 방법의 수는 몇 가지인가?



01

동적 계획법

Dynamic Programming

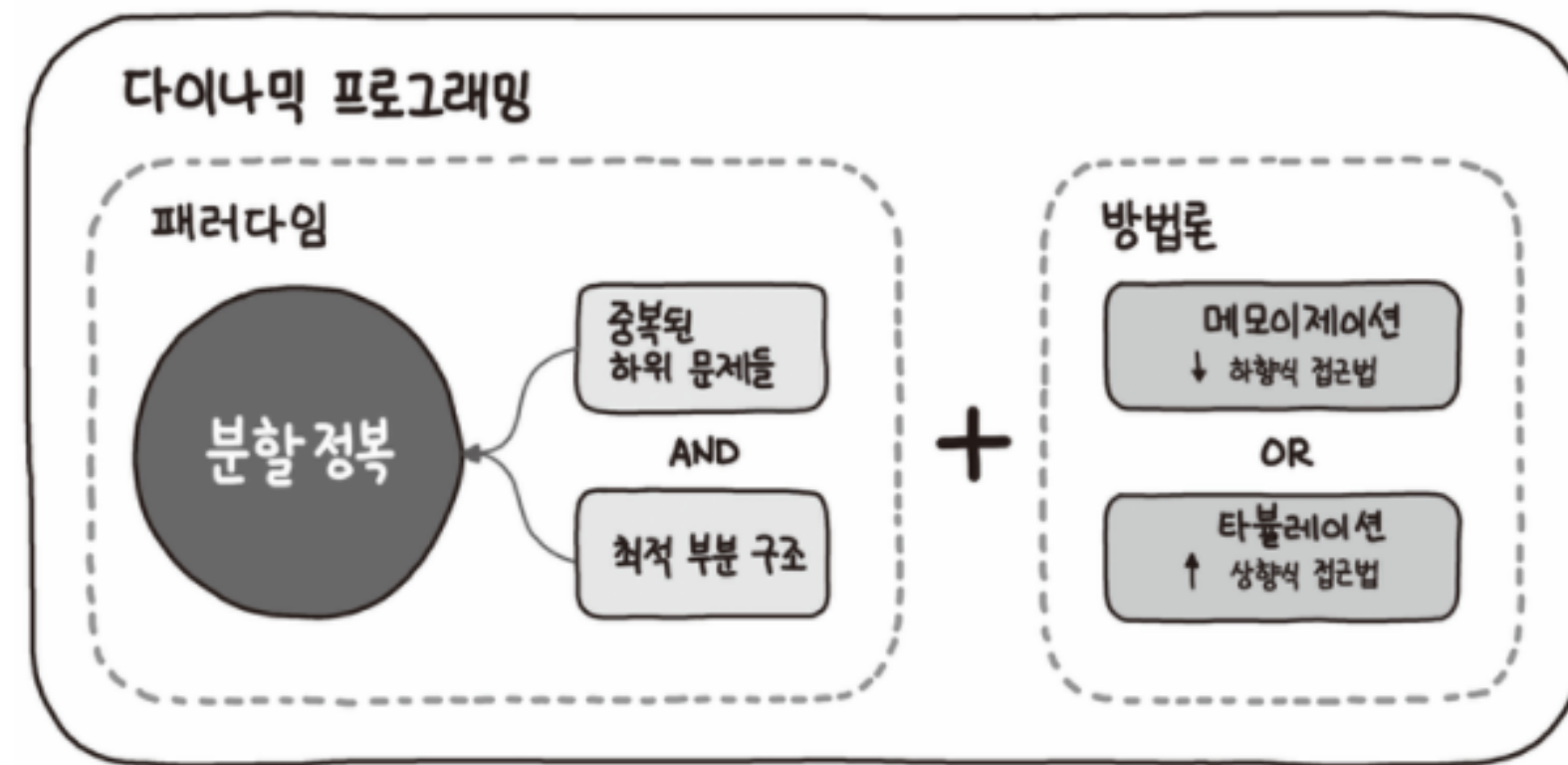
- 각각의 작은 문제로 나누어 해결한 결과를 저장해뒀다가 나중에 큰 문제의 결과와 합하여 풀이하는 알고리즘
- 한 문제가 여러 단계의 반복되는 부분 문제로 이루어졌을 때, 각 단계에 있는 부분 문제의 답을 기반으로 전체 문제의 답을 구하는 방법
- 분할 정복과 달리 한 번 푼 적 있는 부분 문제의 답을 다시 푸는 일이 없도록 테이블 등에 저장해 놓는다.
- 동적 계획법 적용 조건: 최적 부분 구조
 - 전체 문제의 최적해가 부분 문제의 최적해로부터 만들어지는 구조
 - 탐욕 알고리즘도 최적 부분 구조일 때 적용했으나, 동적 계획법은 중복된 하위 문제들의 결과를 저장해뒀다가 풀이해 나간다는 것이 큰 차이점



01

동적 계획법의 설계 Dynamic Programming

- 동적 계획법 알고리즘의 동작 방식 3단계
 - 1) 문제를 부분 문제로 나눈다.
 - 2) 가장 작은 부분 문제부터 해를 구한 뒤 테이블에 저장한다.
 - 3) 테이블에 저장된 부분 문제의 해를 이용하여 점차적으로 상위 부분 문제의 최적해를 구한다.
- 동적 계획법은 분할정복에 중복된 하위 문제들과 최적 부분 구조가 동시에 충족된 경우다.
 - 중복된 하위 문제: 같은 작은 문제가 여러 번 반복 등장(겹치는 하위 문제)
 - 최적 부분 구조: 작은 문제의 최적해로부터 큰 문제의 최적해를 구할 수 있다.
- 동적 계획법 방법론 두 가지
 - 상향식(Bottom-up): 더 작은 하위 문제부터 살펴본 다음, 작은 문제의 정답을 이용해 큰 문제의 정답을 풀어나간다.
 - 타블레이션(Tabulation)이라고도 부르며, 이 방식만을 동적 계획법으로 보기도 한다.
 - 하향식(Top-down): 하위 문제에 대한 정답을 계산했는지 확인해가며 문제를 자연스러운 방식으로 풀어나간다.
 - 메모이제이션(Memoization)이라고도 부른다.



2 동적 계획법 대표 문제

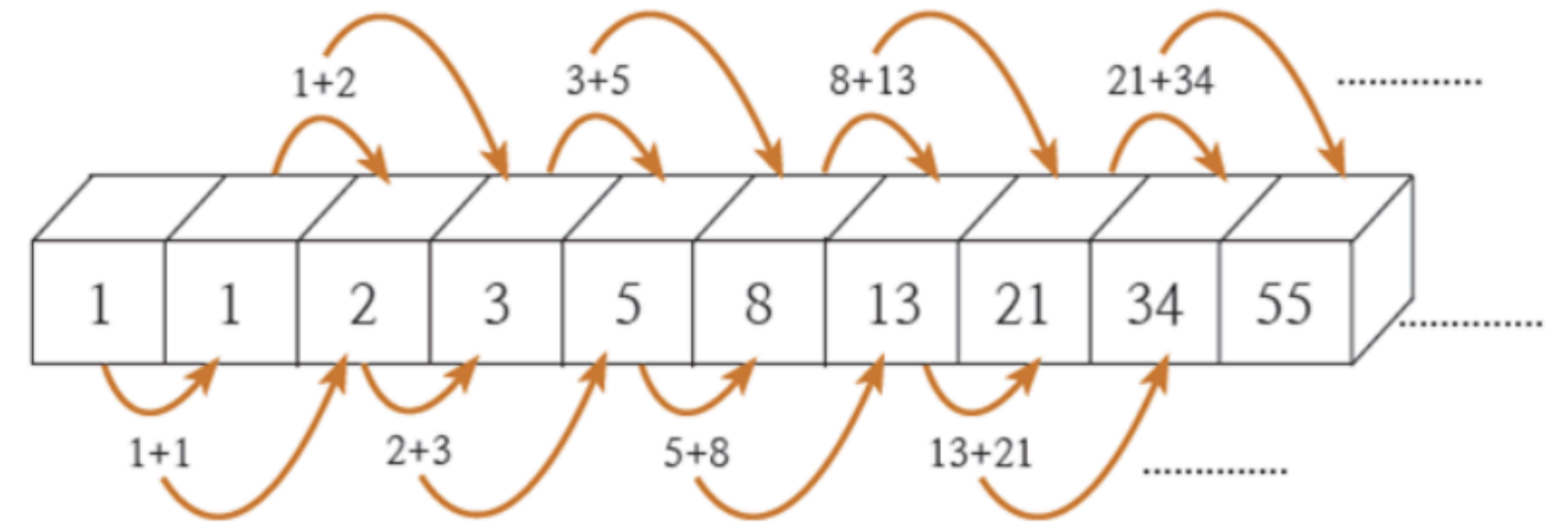
01

피보나치 수열

두 가지 방법론 적용

$$F_n = F_{n-1} + F_{n-2}$$

$$F_1 = F_2 = 1$$



- 상향식 풀이(Tabulation)
 - 작은 하위 문제부터 차례대로 정답을 풀어나가며 큰 문제의 정답을 만든다.
 - 데이터를 테이블(Table) 형태로 만들면서 문제를 풀이한다.

```

1 def fib_bottomup(n):
2     dp = [0] * (n + 1)
3     dp[0] = 0
4     dp[1] = 1
5
6     for i in range(2, n+1):
7         dp[i] = dp[i-1] + dp[i-2]
8     return dp[n]
```

- 하향식 풀이(Memoization)
 - 하위 문제에 대한 정답을 계산했는지 확인해가며 문제를 자연스럽게 재귀로 풀어나간다.
 - 기존 재귀 풀이와 거의 동일하지만 이미 풀어봤는지 확인하며 재사용하는 효율적인 방식

```

1 def fib_topdown(n, memo={}):
2     if n <= 1:
3         return n
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_topdown(n-1, memo) + fib_topdown(n-2, memo)
7     return memo[n]
```


02

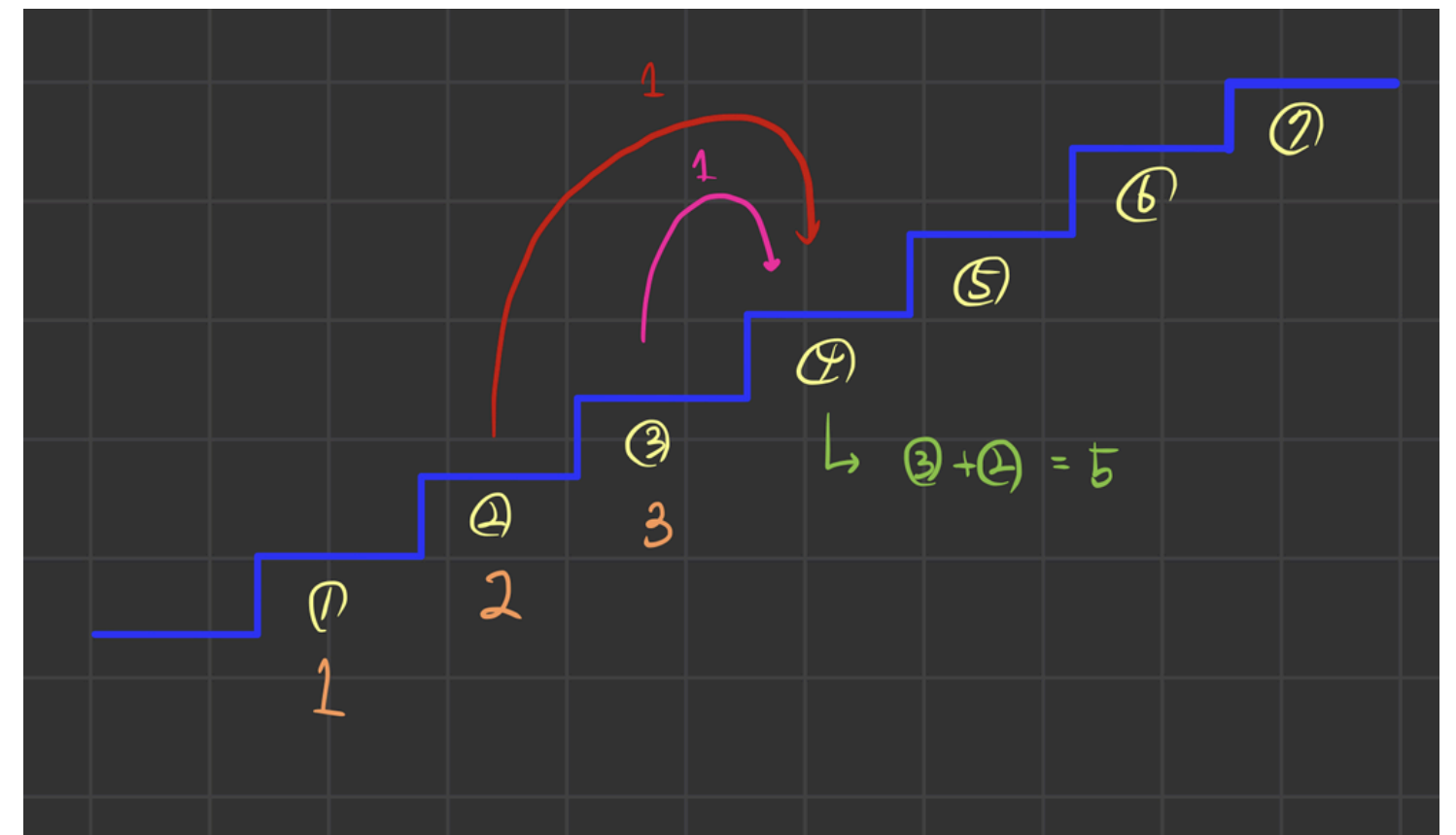
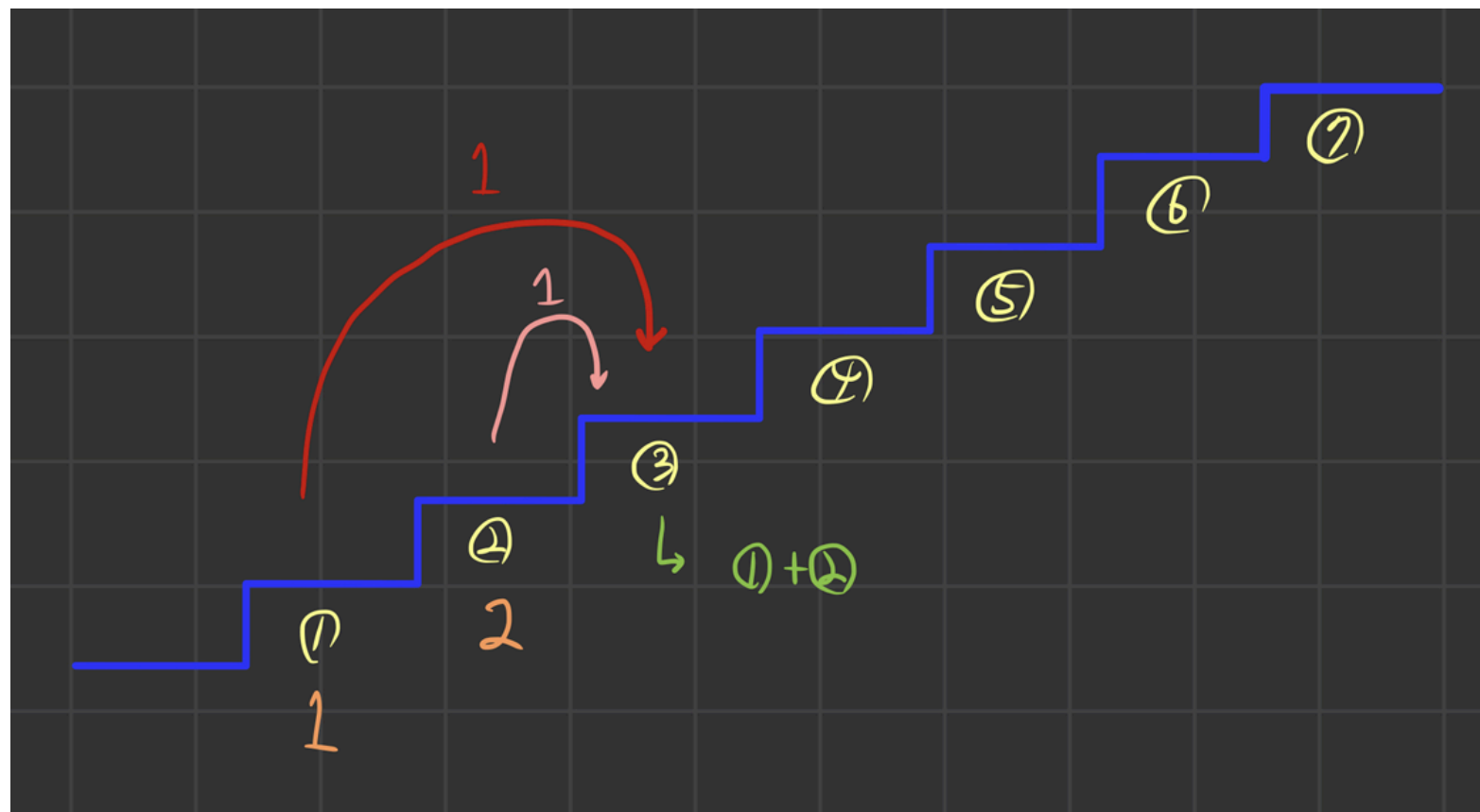
계단 오르기

총 n계단을 오르는 방법의 수는?

철수는 계단을 오를 때 한 번에 한 계단 또는 두 계단씩 올라간다. 만약 총 4계단을 오른다면 그 방법의 수는

1+1+1+1, 1+1+2, 1+2+1, 2+1+1, 2+2 로 5가지이다.

그렇다면 총 N계단일 때 철수가 올라갈 수 있는 방법의 수는 몇 가지인가?

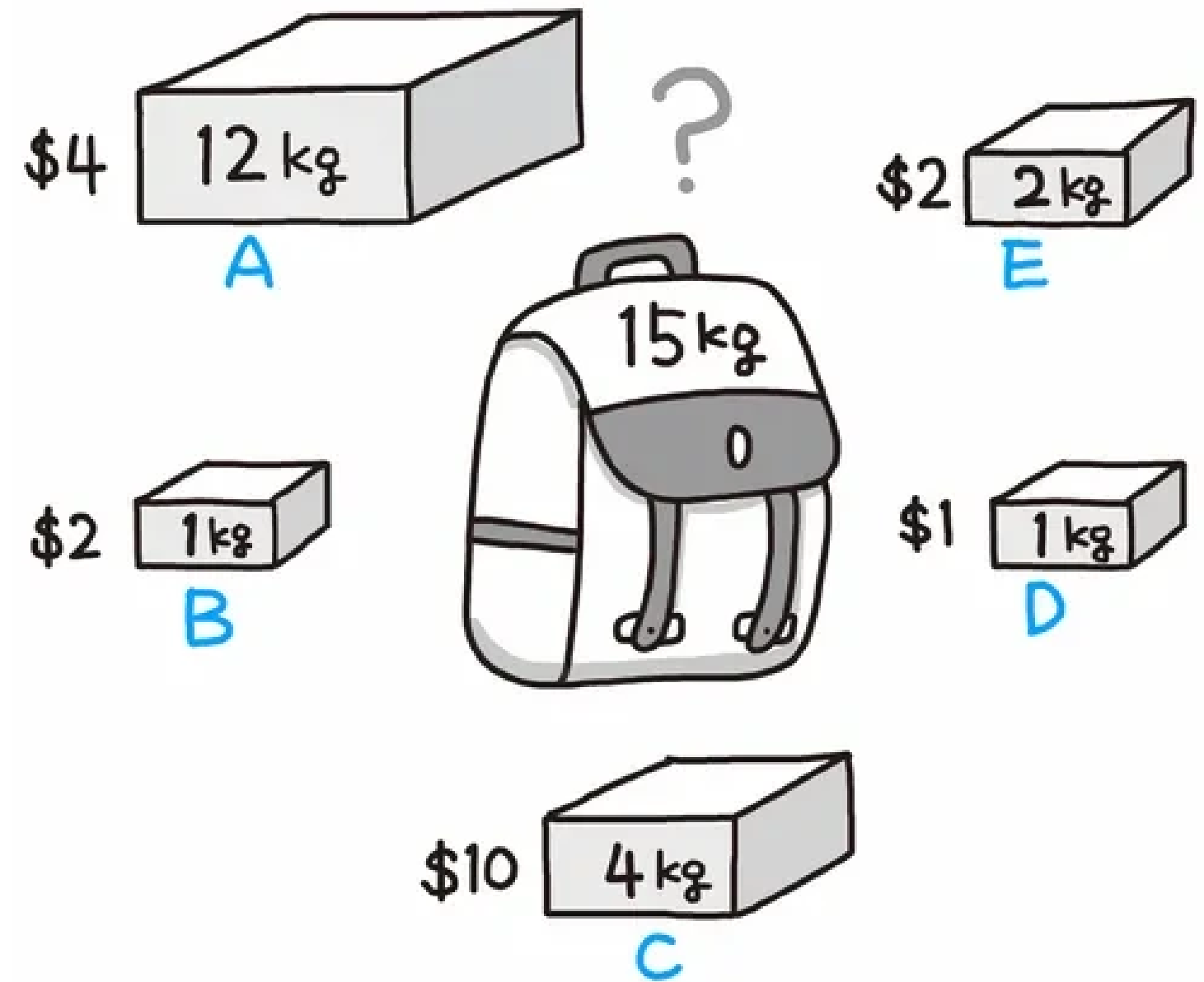


03

0-1 배낭 문제

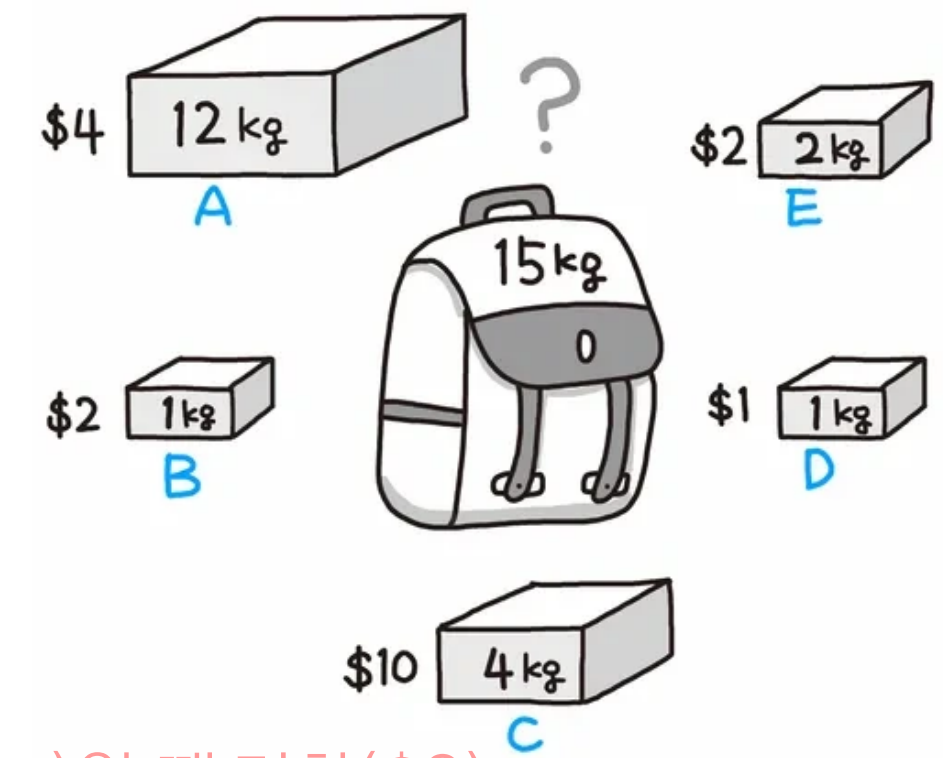
Knapsack problem

- 세종이가 또 도둑질을 하는 상상을 하게 됐다.
- 학교 가는 척 배낭을 메고, 빈 집을 털어 배낭에 최대한 높은 값어치를 담아 유유히 빠져나올 계획이다.
- 세종이 **배낭이 담을 수 있는 최대 무게가 15kg이고, 빈 집에 물건들이 다음과 같이 있을 때, 세종이는 배낭에 무엇을 얼마나 넣어야 할까?** (단, 물건은 자르거나 찢을 수 없다.)



03

0-1 배낭 문제
Knapsack problem



B 추가 X: A만 유지, 가치는 \$4

B 추가 O: B의 가치(\$2) + (현재용량-B무게(1kg)) (11kg)일 때 가치(\$0)

이
위치
까지의
짐의
개수

배낭 용량 짐 개수	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1 A 추가	0	0	0	0	0	0	0	0	0	0	0	0	4	4	4	4
2 B 추가	0	2	2	2	2	2	2	2	2	2	2	2	4	6	6	6
3 C 추가	0	2	2	2	10	12	12	12	12	12	12	12	12	12	12	12
4 D 추가	0	2	3	3	10	12	13	13	13	13	13	13	13	13	13	13
5 E 추가	0	2	3	4	10	12	13	14	15	15	15	15	15	15	15	15

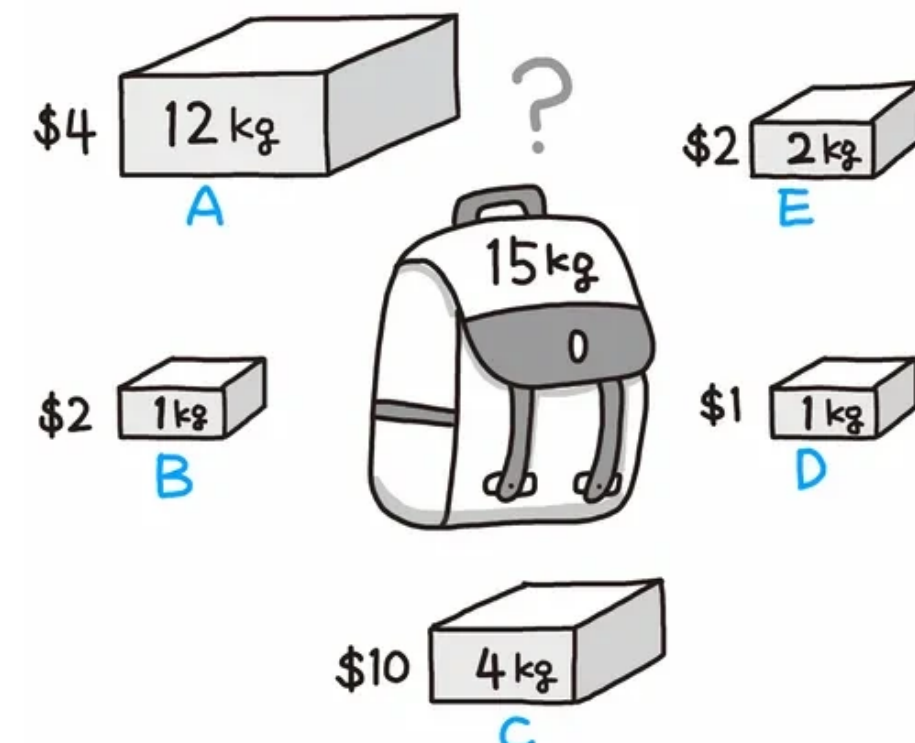
cargo = [(4,12), (2,1), (10,4), (1,1), (2,2)]

03

0-1 배낭 문제

Knapsack problem

```
1 def knapsack(cargo):
2     capa = 15
3     pack=[]
4
5     for i in range(len(cargo)+1):
6         pack.append([])
7         for c in range(capa+1):
8             if i==0 or c==0:
9                 pack[i].append(0)
10            elif cargo[i-1][1] <=c :
11                pack[i].append(max(cargo[i-1][0] + pack[i-1][c-cargo[i-1][1]], pack[i-1][c]))
12            else:
13                pack[i].append(pack[i-1][c])
14     return pack[-1][-1]
```



04

최장 공통 부분 수열

LCS; Longest Common Subsequence

- 부분 수열(subsequence): 수열(객체의 목록)에서 일부 요소를 제거한 것
 - 예) GOOD MORNING → OMIG
- 최장 공통 부분 수열: 두 수열 사이에 공통적으로 존재하는 부분 수열 중 가장 긴 것
 - DNA 염기 서열 비교
 - 텍스트 파일 차이점 찾기

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 , \\ c[i - 1, j - 1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j , \\ \max(c[i, j - 1], c[i - 1, j]) & \text{if } i, j > 0 \text{ and } x_i \neq y_j . \end{cases}$$

04

최장 공통 부분 수열

LCS; Longest Common Subsequence

	i	0	1	2	3	4	5	6
j		-	A	B	C	D	E	F
0	-							
1	G							
2	B							
3	C							
4	D							
5	F							
6	E							

• LCS 테이블

- 문제를 부분 문제로 나눈다.
- 가장 작은 문제부터 해를 구한 뒤 테이블에 저장한다.
- 테이블에 저장된 부분 문제의 해를 이용하여 점차적으로 상위 부분 문제의 최적해를 구한다.

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ c[i - 1, j - 1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j, \\ \max(c[i, j - 1], c[i - 1, j]) & \text{if } i, j > 0 \text{ and } x_i \neq y_j. \end{cases}$$

04

최장 공통 부분 수열

LCS; Longest Common Subsequence

- 빅오는 몇인가?

	i	0	1	2	3	4	5	6
j		-	A	B	C	D	E	F
0	-							
1	G							
2	B							
3	C							
4	D							
5	F							
6	E							

	i	0	1	2	3	4	5	6
j		-	A	B	C	D	E	F
0	-	0	0	0	0	0	0	0
1	G	0	0	0	0	0	0	0
2	B	0	0	1	1	1	1	1
3	C	0	0	1	2	2	2	2
4	D	0						
5	F	0						
6	E	0						

	i	0	1	2	3	4	5	6
j		-	A	B	C	D	E	F
0	-	0	0	0	0	0	0	0
1	G	0	0	0	0	0	0	0
2	B	0	0	1	1	1	1	1
3	C	0	0	1	2	2	2	2
4	D	0	0	1	2	3	3	3
5	F	0	0	1	2	3	3	4
6	E	0	0	1	2	3	4	4

04

최장 공통 부분 수열

LCS; Longest Common Subsequence

1. DP의 마지막 값부터 시작한다.
2. $DP[i-1][j]$ 와 $DP[i][j-1]$ 중 현재 값과 같은 것을 찾는다.
3. 같은 값이 있다면 **해당 값으로 이동**한다.
같은 값이 없다면 **LCS에 해당 문자를 넣고 $DP[i-1][j-1]$ 로 이동**한다.
4. 2번과 3번 과정을 0을 만날 때까지 반복한다.

- $DP[i-1][j]$ 와 $DP[i][j-1]$ 중 현재 값과 같은 값이 있는 경우, 이전 상황과 현재 상황을 비교하였을 때 같은 문자를 찾지 못한 경우이다. 그래서 LCS에 값을 저장하지 않는다.
- $DP[i-1][j]$ 와 $DP[i][j-1]$ 중 현재 값과 같은 값이 없는 경우, 이전 상황과 현재 상황을 비교하였을 때 같은 문자를 찾아서 DP값에 변화가 있었던 경우이다. 그러므로 해당 값을 LCS에 저장해주어야 한다.

	i	0	1	2	3	4	5	6
j		-	A	B	C	D	E	F
0	-	0	0	0	0	0	0	0
1	G	0	0	0	0	0	0	0
2	B	0	0	1	1	1	1	1
3	C	0	0	1	2	2	2	2
4	D	0	0	1	2	3	3	3
5	F	0	0	1	2	3	3	4
6	E	0	0	1	2	3	4	4

LCS

B	C	D	F
---	---	---	---

3 모듈별 탐구

01

계단 오르기 구현

- 정상까지 n 개의 계단을 올라야 한다.
- 매번 각각 1개 또는 2개 계단씩 오를 수 있다면, 정상에 도달하기 위한 방법은 몇 가지인가?
 - 입력: 정상까지의 계단 수(n)
 - 출력: 방법의 수

02

최장 공통 부분 수열 구현

- 두 개의 수열(문자열)에서 최장 공통 부분 수열을 구하라.
 - 입력: 수열(문자열) 2개
 - 출력: LCS 테이블, LCS

03

최대 서브 배열

- 합이 최대가 되는 연속 서브 배열을 찾아 합을 리턴하라.
- 예)
 - 입력: $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$
 - 출력: 6
- 두 가지 방법으로 각각 구현하시오.
 - 분할정복 알고리즘
 - 동적계획법

04

집 도둑

- 당신은 전문털이범이다. 어느 집에서든 돈을 훔쳐올 수 있지만 경보 시스템 때문에 바로 옆집은 훔칠 수 없고 한 칸 이상 떨어진 집만 가능하다. 각 집에는 훔칠 수 있는 돈의 액수가 입력값으로 표기되어 있다. 훔칠 수 있는 가장 큰 금액을 출력하라.
- 단, 타블레이션을 활용할 것.
- 예1)
 - 입력: [1,2,3,1]
 - 출력: 4
- 예2)
 - 입력: [2,7,9,3,1]
 - 출력: 12

05

박람회 부스 배치

- 박람회를 열려고 한다. 전시장은 3행 N열의 격자 형태로 나뉘어 있으며, 각 칸에는 부스를 설치했을 때 예상되는 방문자 만족도 점수가 적혀 있다. 조건을 만족하도록 부스를 설치했을 때, 설치한 칸의 점수 합이 최대가 되도록 하라. 단, 박람회 동선이 복잡해지지 않도록 다음 조건을 지켜야 한다.
- 배치 조건
 - 각 열에는 반드시 하나 이상의 부스를 설치해야 한다. 모든 구역이 활용되도록 하기 위함이다.
 - 상하좌우로 인접한 두 칸에는 동시에 부스를 설치할 수 없다. 인접한 부스는 줄이 겹치거나 통행을 방해하기 때문이다.
 - 각 칸의 점수는 양수 또는 음수일 수 있다.
 - 점수가 음수인 칸은 설치하면 오히려 불리하지만, 각 열에 최소 하나는 설치해야 하므로 선택해야 할 수도 있다.

- 입력: 열 개수, 각 자리의 만족도 점수
- 출력: 부스 배치 결과 만족도 총합

합법적인 예

6	7	12	-5	5	3	11	3
-8	10	14	9	7	13	8	5
11	12	7	4	8	-2	9	4

합법적이지 않은 예

6	7	12	-5	5	3	11	3
-8	10	14	9	7	13	8	5
11	12	7	4	8	-2	9	4

Violation!

06

상점 받기 게임

- 기숙사 명령퇴사 직전, 세종이는 게임 결과로 얻는 점수만큼 상점을 받기로 했다.
- 게임에서 점수를 계산하기 위한 카드가 일렬로 놓여 있는데, 점수 카드와 연산 카드가 번갈아 등장한다.
 - 점수 카드: 자연수가 적힌 카드 / 예) 1,3,8
 - 연산 카드: + 또는 -가 적힌 카드
- 카드의 순서는 바꿀 수 없지만, 인접한 카드들을 하나의 묶음으로 먼저 계산할 수 있고, 묶는 방법에 따라 최종 점수가 달라질 수 있다.
- 가능한 모든 묶는 방법 중, 최종 점수가 가장 크게 나오도록 계산했을 때의 값을 출력하라.

입출력 예

arr	result
["1", "-", "3", "+", "5", "-", "8"]	1
["5", "-", "3", "+", "1", "+", "2", "-", "4"]	3

입출력 예시

입출력 예 #1

위의 예시와 같이 $(1-(3+(5-8))) = 1$ 입니다.

입출력 예 #2

$(5-(3+((1+2)-4))) = 3$ 입니다.

4 동적계획법 정리

01

동적 계획법의 특징
Dynamic Programming

	탐욕 알고리즘	분할 정복	동적 계획법
최적 부분 구조			
중복된 하위 문제들			
탐욕 선택 속성			

01

동적 계획법의 특징
Dynamic Programming

	탐욕 알고리즘	분할 정복	동적 계획법
최적 부분 구조	0	0	0
중복된 하위 문제들			0
탐욕 선택 속성	0		

- 동적 계획법은 큰 문제를 작은 문제로 나누고, 작은 문제의 답을 저장하여 재사용한다.
- 같은 계산을 중복할 필요가 있을 때, 저장을 통해 효율성을 높인다.
- 분할정복은 부분 문제들이 독립적이었지만,
- 동적 계획법에서 부분 문제들은 중복된다.

01

동적 계획법의 특징

Dynamic Programming

- 장점
 - 이미 계산한 결과를 저장하여 재사용하므로 중복 계산을 제거할 수 있다.
 - 불필요한 반복 연산을 줄여 재귀 기반 알고리즘보다 시간 복잡도를 크게 개선할 수 있다.
 - 가능한 모든 경우를 체계적으로 고려하면서도 효율적으로 최적해를 구할 수 있다.
- 단점
 - 문제를 적절한 상태(State)와 점화식으로 표현하는 과정이 어렵다.
 - 계산 결과를 저장하기 위한 추가 메모리가 필요하므로 공간 복잡도가 증가할 수 있다.
 - 최적 부분 구조와 중복 부분 문제가 존재하지 않거나 점화식을 도출하지 못하면 적용하기 어렵다.

02

메모이제이션

Memoization

- memoization: 이전에 계산한 결과를 메모리에 저장하여, 동일한 계산을 반복하지 않고 재사용한다. (memo)
- 메모이제이션 리스트에 값이 저장되어 있으면 다시 계산하지 않는다.
- 계단오르기 by memoization
- memoization과 tabulation

memoization	tabulation
top-down	bottom-up
재귀 사용	반복문 사용
필요한 상태만 계산	작은 문제부터 순서대로 모두 계산

02

메모이제이션

Memoization

- memoization: 이전에 계산한 결과를 메모리에 저장하여, 동일한 계산을 반복하지 않고 재사용한다. (memo)
- 메모이제이션 리스트에 값이 저장되어 있으면 다시 계산하지 않는다.
- 계단오르기 by memoization
- memoization과 tabulation

memoization	tabulation
top-down	bottom-up
재귀 사용	반복문 사용
필요한 상태만 계산	작은 문제부터 순서대로 모두 계산

```
1 n = 10
2
3 memo = [-1] * (n + 1)
4 memo[1] = 1
5 memo[2] = 2
6
7 def climb(n):
8     if memo[n] != -1:
9         return memo[n]
10
11     memo[n] = climb(n-1) + climb(n-2)
12     return memo[n]
13
14 print(climb(n))
```

03

동적 계획법 문제의 풀이

- 동적 계획법 문제일까?
 - 경우의 수를 세는 문제
 - 최댓값/최솟값을 구하는 문제
 - 이전 상태가 현재 상태에 영향을 주는 문제
 - 재귀로 풀면 같은 계산이 반복되는 문제 등
- 동적 계획법 풀이의 핵심 흐름
 - 문제 정의 → 점화식 세우기 → 저장 구조 만들기 → 계산 구현

Q&A
